

Reflexión sobre el trabajo realizado

Student-1, Luis Esteban Acevedo Ladino, 202521217, le.acevedo@uniandes.edu.co

Student-2, Santiago Tarazona Upegui, 202513771, s.tarazonau@uniandes.edu.co

Student-3, Nicolas Prieto Bonilla, 202512129, n.prietob@uniandes.edu.co

Student-1, Yury Dayana Velasquez Alvarez, 202216733, y.velasquez@uniandes.edu.co

Implementación y formulación de los algoritmos

Ascenso a la colina

Representación y formulación del problema

Tabla 1. Formulación del problema ascenso a la colina

Elemento	Descripción
Representación	Configuración binaria ($x_1, x_2, \dots, x_n$) donde $x_i = 1$ significa que se instala un módulo en ese sitio i y $x_i = 0$ significa que no se instala nada en i .
Restricción de validez	La configuración debe tener exactamente k módulos instalados, es decir que la suma de sus bits debe ser igual a k .
Función objetivo	Maximizar el puntaje: cobertura - redundancia - exposición
Vecindad	Se obtiene removiendo un módulo de un sitio específico y colocándolo en un sitio que no estaba seleccionado, manteniendo exactamente la cantidad de k módulos.

Ejemplos de configuraciones iniciales de *local_peak* y *plateau_grid*

Tabla 2. Configuraciones iniciales *local_peak*

Layout	Semilla	Puntaje final	Cobertura	Redundancia	Exposición	Evaluaciones
local_peak	7	96.00	96.00	0.00	0.00	46
local_peak	8	81.00	89.00	0.00	8.00	46

Las dos ejecuciones utilizan el mismo algoritmo y el mismo layout, pero parte de configuraciones iniciales diferentes. La ejecución con semilla 7 alcanza un puntaje de 96.00, mientras que la ejecución con semilla 8 termina con un puntaje de 81.00. Para ambos casos, el algoritmo finaliza por ausencia de una mejora, esto evidencia que ascenso de colina puede alcanzar diferentes soluciones dependiendo de la semilla.

Tabla 3. Configuraciones iniciales *plateau_grid*

Layout	Semilla	Puntaje final	Cobertura	Redundancia	Exposición	Evaluaciones
plateau_grid	4	94.00	96.00	0.00	2.00	31
plateau_grid	5	92.00	96.00	0.00	4.00	46

Se observa que al igual que la ejecución con el layout anterior, diferentes configuraciones iniciales producen distintos puntajes finales, aunque el algoritmo usa el mismo criterio de búsqueda.

Recocido Simulado

Comparación temperatura inicial y factor de enfriamiento

Se compararon tres configuraciones (T_0 , α) con un máximo de 500 iteraciones en *risk_corridor* y *plateau_grid*, tomando ascenso de colina (HC) como referencia. Cada configuración se ejecutó con 30 semillas para distinguir el comportamiento consistente de una corrida particular.

Tabla 4. Comparación datos de prueba recocido simulado

Layout	Algoritmo (T_0 , α)	Mejor	Promedio	Desv.	Mínimo	Veces óptimo	Evaluaciones
risk_corridor	HC (referencia)	116	112,67	3,73	106	15/30	132
risk_corridor	SA (2; 0,90)	116	111,13	3,90	106	9/30	205
risk_corridor	SA (20; 0,97)	116	114,50	2,50	108	20/30	501
risk_corridor	SA (50; 0,99)	116	115,80	0,60	113	26/30	501
plateau_grid	HC (referencia)	94	92,93	1,00	92	14/30	36
plateau_grid	SA (2; 0,90)	94	93,80	0,60	92	27/30	205
plateau_grid	SA (20; 0,97)	94	94,00	0,00	94	30/30	501
plateau_grid	SA (50; 0,99)	94	94,00	0,00	94	30/30	501

Con $T_0 = 2$ y $\alpha = 0,90$ la temperatura cae por debajo del mínimo (10^{-9}) en unas 204 iteraciones. El algoritmo casi no acepta deterioros y se comporta de forma parecida a ascenso de colina: en *risk_corridor* alcanza el óptimo en 9 de 30 corridas (HC lo logra en 15), aunque en *plateau_grid* sí supera a HC (27/30 frente a 14/30) porque le basta aceptar unos pocos movimientos que no mejoran para salir de la meseta.

Al subir la temperatura y enfriar más despacio, (20; 0,97) y (50; 0,99), el algoritmo acepta más deterioros al principio, explora más regiones y se estabiliza al final. En *risk_corridor* el número de corridas óptimas pasa de 9 a 20 y a 26 de 30, y la desviación baja de 3,90 a 0,60. El costo es un mayor número de evaluaciones (501, que es el límite de iteraciones, frente a 205).

Algoritmo Genético

Pruebas con distintos parámetros

Se probó el algoritmo genético en *risk_corridor* y *balanced_city* variando tamaño de población, generaciones, probabilidad de mutación y elitismo. Cada fila resume 30 semillas.

Tabla 5. Algoritmo genético según sus parámetros (30 semillas)

Layout	Población	Generaciones	Mutación	Élite	Promedio	Desv.	Veces óptimo	Evaluaciones
risk_corridor	10	20	0,05	2	109,33	5,00	5/30	170
risk_corridor	40	100	0,05	2	115,60	1,02	26/30	3840
risk_corridor	40	100	0,00	2	115,30	1,44	20/30	3840
risk_corridor	40	100	0,30	2	115,67	1,35	28/30	3840
risk_corridor	100	100	0,05	2	116,00	0,00	30/30	9900
risk_corridor	40	100	0,05	0	115,43	1,80	27/30	4040
balanced_city	10	20	0,05	2	274,60	4,37	0/30	170
balanced_city	40	100	0,05	2	281,73	1,34	14/30	3840
balanced_city	40	100	0,00	2	280,87	2,11	11/30	3840
balanced_city	40	100	0,30	2	282,23	1,26	21/30	3840
balanced_city	100	100	0,05	2	282,80	0,60	27/30	9900
balanced_city	40	100	0,05	0	282,40	1,08	22/30	4040

Población y generaciones: con población 10 y 20 generaciones (170 evaluaciones) el algoritmo casi nunca llega al óptimo (5/30 en *risk_corridor* y 0/30 en *balanced_city*) entonces hay poca diversidad y pocas generaciones para que la selección actúe. Con población 100 se alcanza el óptimo en 30/30 y 27/30 corridas, pero con 9900 evaluaciones, 2,6 veces las de la configuración por defecto.

Mutación: sin mutación la diversidad depende solo del cruce y de la reparación, y el desempeño empeora (20/30 frente a 26/30 en *risk_corridor*; 11/30 frente a 14/30 en *balanced_city*). Con mutación 0,30 mejora (28/30 y 21/30), porque el intercambio de un sitio activo por uno inactivo mantiene la población explorando.

Elitismo: quitar el elitismo (élite = 0) no empeoró los resultados, y en *balanced_city* incluso fue mejor (22/30 frente a 14/30). Como el algoritmo retorna al mejor individuo de toda la ejecución, no necesita el elitismo para no perder la mejor solución.

Minimax

Formulación del problema

Tabla 6. Formulación conceptual de Minimax

Elemento	Descripción
Estado inicial	Se obtiene del layout, incluyendo las posiciones iniciales del defensor y del intruso y el conjunto de terminales pendientes.
Jugador en turno	El defensor (agente 0, MAX) inicia la partida y los jugadores alternan turnos.
Acciones	North, South, East, West y Stop, respetando los límites y obstáculos del layout.
Sucesor	Se genera mediante <code>state.generate_successor(agent_index, action)</code> .
Estado terminal	<code>state.is_win()</code> cuando todas las terminales han sido activadas, o <code>state.is_lose()</code> cuando el intruso alcanza al defensor.
Utilidad	+1000 para una victoria del defensor y -1000 para una derrota.

Poda alfa-beta

Justificación de las características y los pesos de la función de evaluación

La función de evaluación busca distinguir qué tan favorable es un estado no terminal para el defensor. Para ello, se consideran principalmente la cantidad de terminales pendientes, la distancia del defensor a las terminales, la distancia entre el defensor y el intruso, el riesgo inmediato de captura y la movilidad disponible. Estos elementos permiten representar tanto el avance hacia el objetivo como el nivel de seguridad del defensor.

Los factores relacionados con las terminales deben tener un peso importante, ya que el objetivo principal es completar su activación. La cercanía del intruso también debe generar una penalización considerable, especialmente cuando existe riesgo inmediato de captura. Por otro lado, una mayor movilidad puede recibir una recompensa menor, pues disponer de más acciones legales ofrece alternativas para avanzar o escapar, pero no es tan importante como activar las terminales o evitar una derrota.

Los pesos se deben escoger de manera que estos criterios mantengan una jerarquía clara: evitar la captura, avanzar hacia las terminales y, de forma secundaria, conservar movilidad. Además, toda evaluación no terminal debe permanecer estrictamente entre -1000 y +1000, garantizando que una victoria con valor +1000 o una derrota con -1000 siempre tenga prioridad sobre cualquier valoración heurística.

Análisis de los algoritmos y resultados

a) ¿Qué características de los layouts explican las diferencias observadas entre ascenso de colina, recocido simulado y algoritmo genético?

En *local_peak* definitivamente hay máximos locales, porque el ascenso de colina llegó al óptimo (96) en 22 de 30 semillas y en las demás quedó en soluciones como 81 (semilla 8), porque ningún vecino inmediato mejora estrictamente. En *plateau_grid* hay mesetas como HC solo 14 alcanzaron el óptimo (94). El recocido con $T_0 = 20$ y el genético con mutación 0,30 llegaron a 30/30 porque pueden aceptar movimientos que no mejoran (recocido) o explorar varias regiones a la vez (genético).

En *risk_corridor* la cobertura y la exposición los sitios que cubren más son también los de mayor riesgo, lo que crea algunos problemas: HC alcanzó el óptimo en 15/30 corridas, el recocido (50; 0,99) en 26/30 y el genético por defecto en 26/30. En *balanced_city* (16 sitios y 8008 configuraciones) con los parámetros por defecto, todas las corridas quedan a 5 puntos o menos del óptimo (283) y las diferencias entre algoritmos son pequeñas (HC 16/30, recocido 13 a 17/30, genético por defecto 14/30). Solo el genético con población 100 acerca la mayoría de corridas al óptimo (27/30).

b) ¿Qué efecto tuvieron la inicialización, la aleatoriedad y los parámetros principales sobre los resultados de los algoritmos de optimización?

El ascenso en colina es determinista una vez fijada la configuración inicial, así que su resultado depende del seed. En *local_peak* la semilla 7 llega a 96 y la 8 se queda en 81, y hay una desviación de 6,63 sobre las 30 semillas. Con el recocido se reduce esta dependencia porque puede salir de un óptimo local en *local_peak* la desviación baja de 3,71 con (20; 0,97) a 0,00 con (50; 0,99), y el genético la reduce todavía más al mantener una población (desviación 0,00 con población 40 en *local_peak*).

Sin embargo, la aleatoriedad hace que no sea suficiente una sola ejecución. Por ejemplo, el recocido en *risk_corridor* con la semilla 7 dio 109 mientras que su promedio en 30 semillas es 114,50. Los parámetros controlan el balance entre exploración y explotación: en recocido, un enfriamiento muy rápido ($T_0 = 2$; $\alpha = 0,90$) lo deja casi igual a ascenso de colina, mientras que $T_0 = 50$ y $\alpha = 0,99$ explora más; en el genético, la población y las generaciones definen cuánto espacio se cubre, la mutación conserva la diversidad, y el elitismo tuvo un efecto poco claro.

c) ¿Qué compromiso observa entre calidad de la solución y costo computacional en los algoritmos de optimización?

Tabla 7. Compromiso calidad vs costo computacional

Algoritmo	Promedio	Veces óptimo	Evaluaciones	Tiempo (ms)
Ascenso de colina	112,67	15/30	132	< 1
Recocido (50; 0,99)	115,80	26/30	501	6,2
Genético (pob. 40, 100 gen.)	115,60	26/30	3840	37,1
Genético (pob. 100, 100 gen.)	116,00	30/30	9900	95,8

El ascenso de colina tiene menor costo pero resulta ser menos confiable. El recocido (50;0,99) alcanza la misma tasa de éxito que el genético (*default*) pero con 7.7 veces menos evaluaciones y 6 veces menos tiempo. El genético sólo logra 30/30 con una población de 100. La calidad mejora pero resulta ser mucho más costoso. En *balanced_city* ocurre algo similar: HC usa 229 evaluaciones (16/30) y el genético con población 100 usa 9900 (27/30). El costo es en evaluaciones de la función objetivo, que en este problema es barata; si fuera costosa, el compromiso favorecería más al recocido.

d) Identifique un caso en el que cambiar la profundidad modifique la decisión o el resultado del defensor. Explique por qué ocurre y relacione su respuesta con la función de evaluación y el efecto horizonte.

Tabla 8. Resultados de Minimax según profundidad y layout

Layout	Prof.	Acción	Resultado	Nodos	Tiempo (s)
<i>single_terminal</i>	2	East	Victoria	112	0.002995
<i>single_terminal</i>	4	East	Victoria	1.153	0.012492
<i>tiny_defense</i>	2	South	Límite	634	0.009817
<i>tiny_defense</i>	4	South	Límite	12.769	0.140682
<i>bottleneck</i>	2	West	Victoria	260	0.005599
<i>bottleneck</i>	4	West	Victoria	2.192	0.028476
<i>deceptive_distance</i>	2	North	Límite	668	0.009977
<i>deceptive_distance</i>	4	North	Victoria	4.437	0.053787

Al aumentar la profundidad de búsqueda de 2 a 4 *plies*, el número de nodos evaluados aumenta los layouts analizados. En *single_terminal*, pasa de 112 a 1.153 nodos; en *tiny_defense*, de 634 a 12.769; y en *bottleneck*, de 260 a 2.192. Esto muestra el incremento del costo computacional asociado con ampliar el horizonte de búsqueda. El resultado de la partida no necesariamente cambia al aumentar la profundidad. En *single_terminal* y *bottleneck*, el agente alcanza la victoria tanto con profundidad 2 como con profundidad 4, manteniendo además la misma acción inicial (*East* y *West*, respectivamente). En *tiny_defense*, en cambio, ambas profundidades terminan por límite de rondas y mantienen la acción inicial *South*.

Un caso en el que el cambio de profundidad modifica el resultado es *deceptive_distance*. Con profundidad 2, el defensor, al igual que con profundidad 4, selecciona *North*, pero la partida termina por límite de rondas. Esto sugiere que, al aumentar la profundidad a 4, mantiene la misma acción inicial, pero consigue la activación completa de las terminales. Este fenómeno ocurre porque con mayor profundidad Minimax puede anticipar más secuencias de movimientos del defensor y respuestas del intruso antes de aplicar la función de evaluación. Con profundidad 2, algunas de las consecuencias futuras de las acciones quedan fuera del horizonte de búsqueda y los estados de corte se valoran utilizando únicamente la información disponible hasta ese punto. Al utilizar 4 *plies*, el algoritmo puede considerar consecuencias más lejanas y encontrar una secuencia que conduce finalmente a la victoria. Este comportamiento se relaciona directamente con el efecto horizonte, en el que una profundidad de búsqueda limitada puede ocultar consecuencias relevantes que aparecen después del límite de búsqueda.

- e) Explique por qué alfa-beta debe producir la misma decisión que Minimax bajo las mismas condiciones. Analice cómo la profundidad afecta el costo de la búsqueda y cómo el orden de las acciones afecta la eficacia de la poda alfa-beta.

Tabla 9. Resultados de poda alfa-beta

Layout	Prof.	Acción (ambos)	Resultado	Nodos MM	Nodos A-B	Reducción
single_terminal	2	South	Límite	651	428	34,3 %
single_terminal	4	South	Victoria	1.357	410	69,8 %
tiny_defense	2	North	Límite	737	439	40,4 %
tiny_defense	4	North	Límite	10.234	2.479	75,8 %

bottleneck	2	West	Límite	653	417	36,1 %
bottleneck	4	West	Límite	7.047	2.137	69,7 %
deceptive_distance	2	North	Límite	760	460	39,5 %
deceptive_distance	4	North	Límite	10.788	2.244	79,2 %

La poda alfa-beta debe producir la misma decisión que Minimax cuando ambos trabajan bajo las mismas condiciones, ya que no modifica el cálculo del valor Minimax, sino que evita explorar ramas que no pueden afectar la decisión final. Esto se comprobó en las ocho configuraciones analizadas, donde ambos algoritmos seleccionaron la misma acción inicial y obtuvieron el mismo resultado.

La principal diferencia está en el costo de búsqueda. Con profundidad 2, Alpha-Beta redujo los nodos explorados entre 34,3 % y 40,4 % frente a Minimax. Con profundidad 4, la reducción fue aún mayor, entre 69,7 % y 79,2 %, mostrando que la poda resulta especialmente útil cuando aumenta el tamaño del árbol de búsqueda. Aumentar la profundidad permite analizar más movimientos futuros, pero incrementa considerablemente el número de estados explorados. Por ejemplo, en *deceptive_distance*, Minimax pasó de 760 a 10.788 nodos, mientras Alpha-Beta pasó de 460 a 2.244. Por lo tanto, una mayor profundidad sigue aumentando el costo, pero Alpha-Beta reduce significativamente este crecimiento.

En *single_terminal*, Alpha-Beta exploró menos nodos totales con profundidad 4 que con profundidad 2 (410 frente a 428) porque con profundidad 4 obtuvo la victoria en 9 rondas, mientras que con profundidad 2 llegó al límite de 40 rondas. Por ello, los nodos acumulados de una partida también dependen de su duración.

Reflexión del uso de IA

Durante este taller usamos la IA principalmente como apoyo para revisar, depurar y comprender las implementaciones realizadas. Inicialmente se dispuso de esta para tener en cuenta las divisiones claras del taller y sus puntos, para así poder dividir el trabajo equitativamente, de esta manera se tuvo un mejor entendimiento de las fuertes divisiones y aspectos parecidos entre los algoritmos en tema de código.

En materia de código, la IA resultó especialmente útil durante el proceso de depuración. A medida que avanzábamos fueron apareciendo errores y dudas puntuales relacionadas con la implementación, por lo que recurrimos a ella para revisar fragmentos específicos, entender por

qué se estaban produciendo ciertos resultados y plantear posibles soluciones. Por ejemplo, en la parte de búsqueda adversaria nos ayudó a comprender mejor el funcionamiento de la poda alfa-beta, las condiciones de corte y la forma de verificar que sus decisiones fueran consistentes con las de Minimax.

En conclusión, la IA fue una herramienta de apoyo durante el desarrollo del taller, principalmente para aclarar dudas, encontrar errores y comprender mejor algunos conceptos. Sin embargo, todas las sugerencias fueron revisadas y verificadas mediante pruebas antes de incluirlas en el repositorio. Esto nos permitió no solo avanzar en la implementación, sino también entender el funcionamiento de los algoritmos utilizados y estar en la capacidad de explicar las decisiones tomadas gracias a la ayuda de la IA.